
📘 Course Project Assignment: Software Design & Architecture Challenge

Duration: 2 Weeks
Team Size: 3–5 students
Due Date: May 1st, 2025

🎯 Objective

You've now studied a wide array of essential software design principles, patterns, and practices—from OOP and SOLID principles to architectural styles and exception handling best practices. For this capstone-style assignment, your task is to **design and build a software application** in teams that thoughtfully incorporates many (if not all) of these concepts.

This is not just about making an application that "works"—it's about creating **well-structured, maintainable, and defensible** software.

🛠️ Project Requirements

Your group must design and implement an application of your choice (e.g., task manager, e-commerce API, chat system, library system, blog CMS, inventory tracker, etc.) that meets the following requirements:

✅ Incorporate the following concepts into your project:

📌 *Design Principles*

- Apply key **Software Design Principles** (e.g., DRY, SoC, Program to Interface)
- Demonstrate **High Cohesion & Low Coupling**
- Emphasize **Abstraction, Encapsulation, Composition over Inheritance**, etc.

📌 *Object-Oriented Programming (OOP)*

- Show effective use of **Encapsulation, Abstraction, Inheritance, Polymorphism**
- Model relationships through **Association, Aggregation, Composition**

- Apply **Method Overloading/Overriding** and thoughtful constructor design

📌 *SOLID Principles*

- Clearly reflect how your design follows **SRP, OCP, LSP, ISP, DIP**

📌 *Design Patterns*

- Implement **at least 3 different design patterns**, from different categories (Creational, Structural, Behavioral)

  For example: Factory + Adapter + Observer

📌 *Exception Handling*

- Use **meaningful custom exceptions**
- Follow best practices like **throw early, catch late**, no empty catch blocks, etc.

📌 *Architecture*

- Choose and apply an **architecture style** (e.g., Layered Architecture, SOA, Microservice-style modularity, or Event-Driven Architecture)
- Be able to **justify your architectural decisions** and explain trade-offs

📌 *Documentation & Diagrams*

- Include at least the following:
  - **UML Class Diagram**
  - **Sequence Diagram** for one major use case
  - (Optional: Activity Diagram, Component Diagram)
- Document how your project applies each core topic in a separate **README or Design Document**

📌 *Performance & Scalability*

- Discuss in your documentation:
    - How you handled or would handle **time/space complexity trade-offs**
    - Any **concurrency concerns or thread-safety** considerations (if applicable)

🧠 Deliverables

1. **Source Code** (GitHub repo or ZIP)
2. **Design Document** (PDF or Markdown, including UML diagrams)
3. **Demo Video or In-Class Presentation** (5–8 minutes)
4. **Group Reflection**: Brief description of each team member's contributions

---

🗣️ Defend Your Design

You should be able to **defend your design choices** in terms of:

- Why you chose certain patterns or principles
- What alternatives you considered
- The **pros and cons** of the decisions you made
- Any trade-offs or compromises for simplicity, performance, or flexibility

---

✅ Grading Rubric Overview

Component	Points
Code quality & functionality	25

Application of design principles (DRY, SoC, etc.)	15
Use of OOP and SOLID	15
Design pattern usage	10
Exception handling quality	10
Architecture clarity and justification	10
UML diagrams and documentation	10
Presentation/defense	5
Total	**100**
